

Analyzing the Role of AI-Assisted Programming in Modern Software Development

S17262

H. A. P. W. PATHIRANA

Task 1: Conceptual Understanding

What is AI-Assisted Programming?

AI-assisted programming involves incorporating AI tools to aid software developers across the software development lifecycle. The tools are intended for developers to write code, create code snippets, detect and correct errors, suggest improvements, explain current code and perform repetitive programming tasks. Instead of programmers, AI is an intelligent assistant to increase productivity and enable developers to concentrate on complex problem-solving.

Current AI for programming involves machine learning and natural language processing to interpret user directions and create relevant code. Developers can write a simple program in natural language. And AI can provide suggestions for the code, which can be edited and added to the application.

Types of AI coding tools

Code Generation Tools

These tools produce source code as a result of user requirements or natural language prompts. They aid in the development of functions, classes, algorithms, and even complete programs in a faster manner.

Examples: ChatGPT

GitHub Copilot

Amazon Code Whisperer

Debugging Tools

AI debugging tools analyze code to identify syntax errors, logical mistakes, and runtime issues. They frequently offer solutions and provide reasons for mistakes.

Code Refactoring

Tools These tools enhance existing code without impacting the code's functionality. They can improve the readability, maintainability, and efficiency of the code by restructuring it.

Code Completion Tools

The tools predict and suggest code to the developer as they type, thereby eliminating repetitive tasks and speeding up code typing.

Documentation Assistants

Projects can be more easily understood and maintained with AI-generated comments, documentation, and explanations of source code.

Advantages in software development

- Improved productivity: AI can handle repetitive coding tasks, enabling developers to complete projects more quickly.
- Enhanced code quality: AI provides best practice recommendations and can highlight typical programming errors.
- Faster debugging: AI can suggest corrections to make developers more efficient in finding and fixing bugs.
- The explanation and examples provided by AI for programming concepts make it easy to understand the concepts for beginners.
- Reduced development time: Code generation and intelligent code completion speed up software development.
- Improve documentation: AI can add comments and documentation, making it easier to maintain.

Risks and limitations

- AI can produce faulty or unworkable code as a correct code.
- Code can contain security vulnerabilities if not thoroughly checked.
- Over-dependence on AI can diminish a developer's problem-solving and coding abilities.
- AI might not fully grasp the project's context or business logic.
- Can provide incorrect recommendations.
- Developers need to address concerns like responsible use of AI-generated content, privacy, licensing, and ownership of the code.

Task 2: Practical Implementation

1. Select a programming problem of moderate complexity

The Student Management System is a console-based Java application designed to help users manage student records efficiently. The system allows users to add new students, view all student records, search for a student, update student information, and delete student records.

2. Develop the solution using TWO approaches:

Version A: Traditional programming approach

1. Student.java
2. StudentManager.java
3. Main.java

```
1 public class Student { // no usages
2
3     // Attributes
4     private String studentId; // 3 usages
5     private String name; // 4 usages
6     private int age; // 4 usages
7     private String course; // 4 usages
8     private double gpa; // 4 usages
9
10    // Constructor
11    public Student(String studentId, String name, int age, String course, double gpa) { // no usages
12        this.studentId = studentId;
13        this.name = name;
14        this.age = age;
15        this.course = course;
16        this.gpa = gpa;
17    }
18
19    // Getters
20    public String getStudentId() { // no usages
21        return studentId;
22    }
23
24    public String getName() { // no usages
25        return name;
26    }
27
28    public int getAge() { // no usages
29        return age;
30    }
31
32    public String getCourse() { // no usages
33        return course;
34    }
35
36    public double getGpa() { // no usages
37        return gpa;
38    }
39
40    // Setters
41    public void setName(String name) { // no usages
42        this.name = name;
43    }
44
45    public void setAge(int age) { // no usages
46        this.age = age;
47    }
48
49    public void setCourse(String course) { // no usages
50        this.course = course;
51    }
52
53    public void setGpa(double gpa) { // no usages
54        this.gpa = gpa;
55    }
56
57    // Display student information
58    public void displayStudent() { // no usages
59        System.out.println("-----");
60        System.out.println("Student ID : " + studentId);
61        System.out.println("Name : " + name);
62        System.out.println("Age : " + age);
63        System.out.println("Course : " + course);
64        System.out.println("GPA : " + gpa);
65    }
66 }
```

Student.java

```
1 import java.util.ArrayList;
2
3 public class StudentManager { // no usages
4
5     private ArrayList<Student> students = new ArrayList<>(); // 3 usages
6
7     // Add Student
8     public void addStudent(Student student) { // no usages
9         students.add(student);
10        System.out.println("Student added successfully!");
11    }
12
13    // View All Students
14    public void viewStudents() { // no usages
15        if (students.isEmpty()) {
16            System.out.println("No student records found.");
17            return;
18        }
19
20        for (Student student : students) {
21            student.displayStudent();
22        }
23    }
24
25    // Search Student
26    public Student searchStudent(String studentId) { // 2 usages
27        for (Student student : students) {
28            if (student.getStudentId().equalsIgnoreCase(studentId)) {
29                return student;
30            }
31        }
32        return null;
33    }
34
35    // Search Student
36    public Student searchStudent(String studentId) { // 2 usages
37        for (Student student : students) {
38            if (student.getStudentId().equalsIgnoreCase(studentId)) {
39                return student;
40            }
41        }
42        return null;
43    }
44
45    // Update Student
46    public boolean updateStudent(String studentId, String name, int age, String course, double gpa) { // no usages
47        Student student = searchStudent(studentId);
48
49        if (student != null) {
50            student.setName(name);
51            student.setAge(age);
52            student.setCourse(course);
53            student.setGpa(gpa);
54            return true;
55        }
56        return false;
57    }
58
59    // Delete Student
60    public boolean deleteStudent(String studentId) { // no usages
61
62        Student student = searchStudent(studentId);
63
64        if (student != null) {
65            students.remove(student);
66            return true;
67        }
68        return false;
69    }
70 }
```

StudentManager.java

```

1 import java.util.Scanner;
2
3 public class Main {
4
5     public static void main(String[] args) {
6
7         Scanner scanner = new Scanner(System.in);
8         StudentManager manager = new StudentManager();
9
10        int choice;
11
12        do {
13            System.out.println("\n===== Student Management System =====");
14            System.out.println("1. Add Student");
15            System.out.println("2. View All Students");
16            System.out.println("3. Search Student");
17            System.out.println("4. Update Student");
18            System.out.println("5. Delete Student");
19            System.out.println("6. Exit");
20            System.out.print("Enter your choice: ");
21
22            choice = scanner.nextInt();
23            scanner.nextLine(); // Clear Buffer
24
25            switch (choice) {
26
27                case 1:
28                    System.out.print("Enter Student ID: ");
29                    String id = scanner.nextLine();
30
31                    System.out.print("Enter Name: ");
32                    String name = scanner.nextLine();
33
34                    System.out.print("Enter Age: ");
35                    int age = scanner.nextInt();
36                    scanner.nextLine();
37
38                    System.out.print("Enter Course: ");
39                    String course = scanner.nextLine();
40
41                    System.out.print("Enter GPA: ");
42                    double gpa = scanner.nextDouble();
43                    scanner.nextLine();
44
45                    Student student = new Student(id, name, age, course, gpa);
46                    manager.addStudent(student);
47                    break;
48
49                case 2:
50                    manager.viewStudents();
51                    break;
52
53                case 3:
54                    System.out.print("Enter Student ID to search: ");
55                    id = scanner.nextLine();
56
57                    Student found = manager.searchStudent(id);
58
59                    if (found != null) {
60                        found.displayStudent();
61                    } else {
62                        System.out.println("Student not found.");
63                    }
64                    break;
65
66                case 4:
67                    System.out.print("Enter Student ID to update: ");
68                    id = scanner.nextLine();
69
70                    System.out.print("Enter New Name: ");
71                    name = scanner.nextLine();
72
73                    System.out.print("Enter New Age: ");
74                    age = scanner.nextInt();
75                    scanner.nextLine();
76
77                    System.out.print("Enter New Course: ");
78                    course = scanner.nextLine();
79
80                    System.out.print("Enter New GPA: ");
81                    gpa = scanner.nextDouble();
82                    scanner.nextLine();
83
84                    if (manager.updateStudent(id, name, age, course, gpa)) {
85                        System.out.println("Student updated successfully.");
86                    } else {
87                        System.out.println("Student not found.");
88                    }
89                    break;
90
91                case 5:
92                    System.out.print("Enter Student ID to delete: ");
93                    id = scanner.nextLine();
94
95                    if (manager.deleteStudent(id)) {
96                        System.out.println("Student deleted successfully.");
97                    } else {
98                        System.out.println("Student not found.");
99                    }
100                   break;
101
102                case 6:
103                    System.out.println("Thank you for using Student Management System.");
104                    break;
105
106                default:
107                    System.out.println("Invalid choice! Please try again.");
108            }
109
110        } while (choice != 6);
111
112        scanner.close();
113    }
114}

```

```

Run Main x
===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 1
Enter Student ID: 1
Enter Name: Hirushi
Enter Age: 24
Enter Course: SPS
Enter GPA: 3.1
Student added successfully!

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice: 2
-----
Student ID : 1
Name      : Hirushi
Age       : 24
Course    : SPS
GPA       : 3.1

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
Enter your choice:

```

Result

Main.java

Version B: AI-assisted programming approach

1. Student.java
2. StudentManager.java
3. Main.java

```
1  /**
2   * Student class represents a student's information.
3   */
4  public class Student {
5      // no usages
6
7      private String studentId;
8      private String name;
9      private int age;
10     private String course;
11     private double gpa;
12
13     /**
14      * Constructor
15      */
16     public Student(String studentId, String name, int age, String course, double gpa) {
17         this.studentId = studentId;
18         this.name = name;
19         this.age = age;
20         this.course = course;
21         this.gpa = gpa;
22     }
23
24     // Getters
25     public String getStudentId() {
26         return studentId;
27     }
28
29     public String getName() {
30         return name;
31     }
32
33     public int getAge() {
34         return age;
35     }
36
37     public String getCourse() {
38         return course;
39     }
40
41     public double getGpa() {
42         return gpa;
43     }
44
45     // Setters
46     public void setName(String name) {
47         this.name = name;
48     }
49
50     public void setAge(int age) {
51         this.age = age;
52     }
53
54     public void setCourse(String course) {
55         this.course = course;
56     }
57
58     public void setGpa(double gpa) {
59         this.gpa = gpa;
60     }
61
62     /**
63      * Display student details.
64      */
65     public void displayStudent() {
66         System.out.println("-----");
67         System.out.println("Student ID : " + studentId);
68         System.out.println("Name       : " + name);
69         System.out.println("Age       : " + age);
70         System.out.println("Course    : " + course);
71         System.out.printf("GPA       : %.2f\n", gpa);
72         System.out.println("-----");
73     }
74 }
75
```

Student.java

```
1  import java.util.ArrayList;
2
3  /**
4   * Manages all student-related operations.
5   */
6  public class StudentManager {
7      // no usages
8
9      // List to store student records
10     private ArrayList<Student> students = new ArrayList<>();
11
12     /**
13      * Adds a new student after checking for duplicate IDs.
14      * @param student Student object to be added
15      * @return true if added successfully, false if Student ID already exists
16      */
17     public boolean addStudent(Student student) {
18         if (searchStudent(student.getStudentId()) != null) {
19             return false; // Duplicate ID found
20         }
21
22         students.add(student);
23         return true;
24     }
25
26     /**
27      * Displays all students.
28      */
29     public void viewStudents() {
30         if (students.isEmpty()) {
31             System.out.println("\nNo student records available.");
32             return;
33         }
34
35         System.out.println("\n===== Student Records =====");
36
37         for (Student student : students) {
38             student.displayStudent();
39         }
40     }
41
42     /**
43      * Searches for a student using Student ID.
44      * @param studentId Student ID
45      * @return Student object if found, otherwise null
46      */
47     public Student searchStudent(String studentId) {
48         for (Student student : students) {
49             if (student.getStudentId().equalsIgnoreCase(studentId)) {
50                 return student;
51             }
52         }
53
54         return null;
55     }
56
57     /**
58      * Updates an existing student's details.
59      * @param studentId Student ID
60      * @param name Name
61      * @param age Age
62      * @param course Course
63      * @param gpa GPA
64      * @return true if update is successful
65      */
66     public boolean updateStudent(String studentId, String name, int age, String course, double gpa) {
67         Student student = searchStudent(studentId);
68
69         if (student == null) {
70             return false;
71         }
72
73         student.setName(name);
74         student.setAge(age);
75         student.setCourse(course);
76         student.setGpa(gpa);
77
78         return true;
79     }
80
81     /**
82      * Deletes a student record.
83      * @param studentId Student ID
84      * @return true if deleted successfully
85      */
86     public boolean deleteStudent(String studentId) {
87         Student student = searchStudent(studentId);
88
89         if (student == null) {
90             return false;
91         }
92
93         students.remove(student);
94         return true;
95     }
96
97     /**
98      * Returns the total number of students.
99      */
100    public int getStudentCount() {
101        return students.size();
102    }
103
104 }
105
106
107
108
109
110
111
112
```

StudentManager.java

```

1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3
4 public class Main {
5
6     public static void main(String[] args) {
7
8         Scanner scanner = new Scanner(System.in);
9         StudentManager manager = new StudentManager();
10
11         int choice = 0;
12
13         do {
14
15             System.out.println("\n===== Student Management System =====");
16             System.out.println("1. Add Student");
17             System.out.println("2. View All Students");
18             System.out.println("3. Search Student");
19             System.out.println("4. Update Student");
20             System.out.println("5. Delete Student");
21             System.out.println("6. Display Total Students");
22             System.out.println("7. Exit");
23             System.out.print("Enter your choice: ");
24
25             // Validate menu input
26             try {
27                 choice = scanner.nextInt();
28                 scanner.nextLine();
29             } catch (InputMismatchException e) {
30                 System.out.println("Invalid input! Please enter a number.");
31                 scanner.nextLine();
32                 continue;
33             }
34
35             switch (choice) {
36
37                 case 1:
38
39                     System.out.print("Enter Student ID: ");
40                     String id = scanner.nextLine();
41
42                     System.out.print("Enter Student Name: ");
43                     String name = scanner.nextLine();
44
45                     // Validate Age
46                     int age;
47
48                     while (true) {
49
50                         try {
51
52                             System.out.print("Enter Age: ");
53                             age = scanner.nextInt();
54
55                             if (age > 0) {
56                                 scanner.nextLine();
57                                 break;
58                             }
59
60                             System.out.println("Age must be greater than 0.");
61
62                         } catch (InputMismatchException e) {
63
64                             System.out.println("Invalid age. Please enter a number.");
65                             scanner.nextLine();
66
67                         }
68                     }
69
70                     System.out.print("Enter Course: ");
71                     String course = scanner.nextLine();
72
73                     // Validate GPA
74                     double gpa;
75                     while (true) {
76
77                         try {
78
79                             System.out.print("Enter GPA (0.0 - 4.0): ");
80                             gpa = scanner.nextDouble();
81
82                             if (gpa >= 0 && gpa <= 4.0) {
83                                 scanner.nextLine();
84                                 break;
85                             }
86
87                             System.out.println("GPA must be between 0.0 and 4.0.");
88
89                         } catch (InputMismatchException e) {
90
91                             System.out.println("Invalid GPA.");
92                             scanner.nextLine();
93
94                         }
95                     }
96
97                 }
98
99             }

```

Main.java

```
99
100
101     Student student = new Student(id, name, age, course, gpa);
102
103     if (manager.addStudent(student)) {
104         System.out.println("Student added successfully!");
105     } else {
106         System.out.println("Student ID already exists.");
107     }
108
109     break;
110
111 case 2:
112     manager.viewStudents();
113     break;
114
115 case 3:
116
117     System.out.print("Enter Student ID: ");
118     id = scanner.nextLine();
119
120     Student found = manager.searchStudent(id);
121
122     if (found != null) {
123         found.displayStudent();
124     } else {
125         System.out.println("Student not found.");
126     }
127
128     break;
129
130 case 4:
131
132     System.out.print("Enter Student ID to update: ");
133     id = scanner.nextLine();
134
135     if (manager.searchStudent(id) == null) {
136         System.out.println("Student not found.");
137         break;
138     }
139
140     System.out.print("Enter New Name: ");
141     name = scanner.nextLine();
142
143     while (true) {
144         try {
145             System.out.print("Enter New Age: ");
146             age = scanner.nextInt();
147
148             if (age > 0) {
149                 scanner.nextLine();
150                 break;
151             }
152
153             System.out.println("Age must be greater than 0.");
154         } catch (InputMismatchException e) {
155
156             System.out.println("Invalid age.");
157             scanner.nextLine();
158         }
159     }
160
161     System.out.print("Enter New Course: ");
162     course = scanner.nextLine();
163
164     while (true) {
165         try {
166             System.out.print("Enter New GPA: ");
167             gpa = scanner.nextDouble();
168
169             if (gpa >= 0 && gpa <= 4.0) {
170                 scanner.nextLine();
171                 break;
172             }
173
174             System.out.println("GPA must be between 0.0 and 4.0.");
175         } catch (InputMismatchException e) {
176
177             System.out.println("Invalid GPA.");
178             scanner.nextLine();
179         }
180     }
181
182     if (manager.updateStudent(id, name, age, course, gpa)) {
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

Main.java

```

196         break;
197
198     case 5:
199
200         System.out.print("Enter Student ID to delete: ");
201         id = scanner.nextLine();
202
203         if (manager.deleteStudent(id)) {
204             System.out.println("Student deleted successfully.");
205         } else {
206             System.out.println("Student not found.");
207         }
208
209         break;
210
211     case 6:
212
213         System.out.println("Total Students : "
214             + manager.getStudentCount());
215
216         break;
217
218     case 7:
219
220         System.out.println("Thank you for using the Student Management System.");
221
222         break;
223
224     default:
225
226         System.out.println("Invalid menu option.");
227
228     }
229
230     } while (choice != 7);
231
232     scanner.close();
233 }
234

```

Main.java

```

"C:\Users\Hirushi Wijenayaka\.jdk\openjdk-26.0.1\bin\java.exe"

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Total Students
7. Exit
Enter your choice: 1
Enter Student ID: 01
Enter Student Name: Parames
Enter Age: 22
Enter Course: Java
Enter GPA (0.0 - 4.0): 3.2
Student added successfully!

===== Student Management System =====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Display Total Students
7. Exit
Enter your choice: 4
Enter Student ID to update: 01
Enter New Name: Adeesha
Enter New Age: 22
Enter New Course: java
Enter New GPA: 3.0
Student updated successfully.

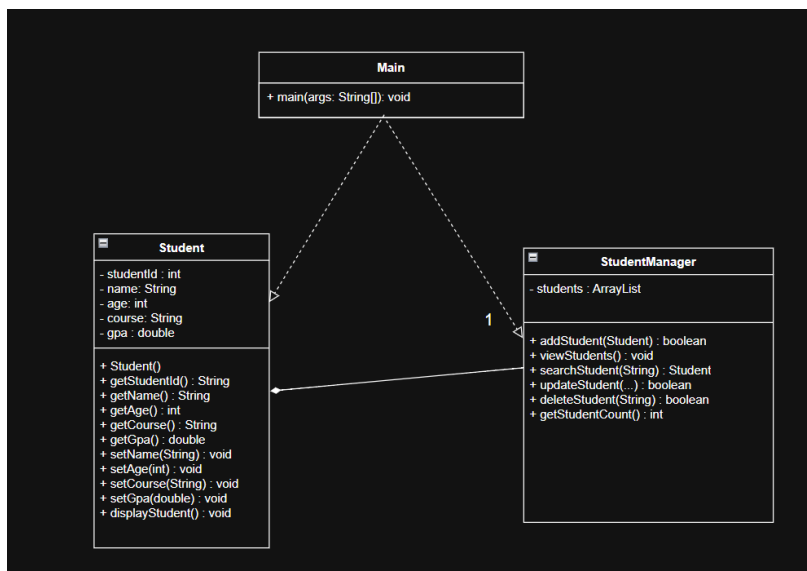
```

Result

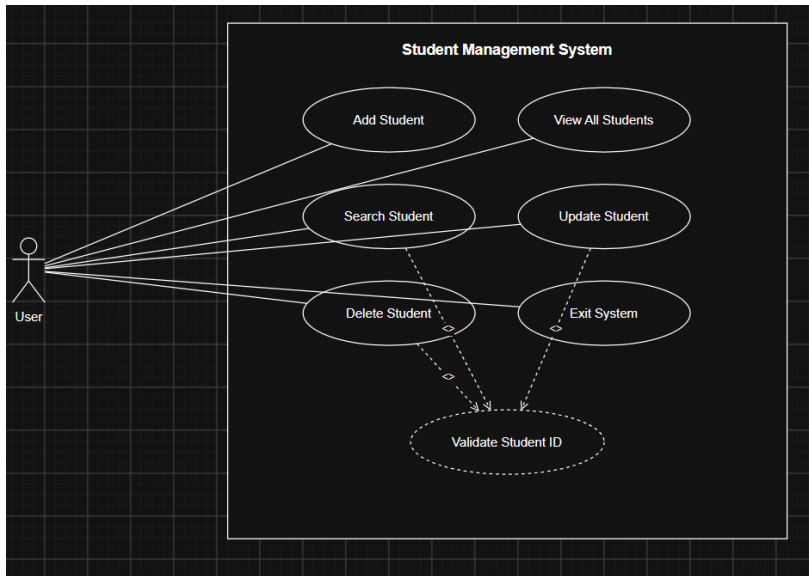
3. Comparison of version A and version B

Criteria	Version A	Version B
Development Time	Development took longer because the code was written manually and required more time for debugging and testing.	Development was faster because AI generated code suggestions, provided solutions for common problems, and reduced coding effort.
Code Quality	Functional and well-structured but contained limited validation and fewer comments.	Improved code quality with better documentation, input validation, duplicate ID checking, and cleaner structure.
Error Handling	Basic error handling with limited protection against invalid user input.	Comprehensive exception handling using try-catch blocks and input validation prevented the application from crashing due to invalid input.
Readability & Maintainability	Code was readable but contained minimal comments and documentation.	Code included Java Doc comments, meaningful method descriptions, improved formatting, and reusable methods, making it easier to maintain and understand.

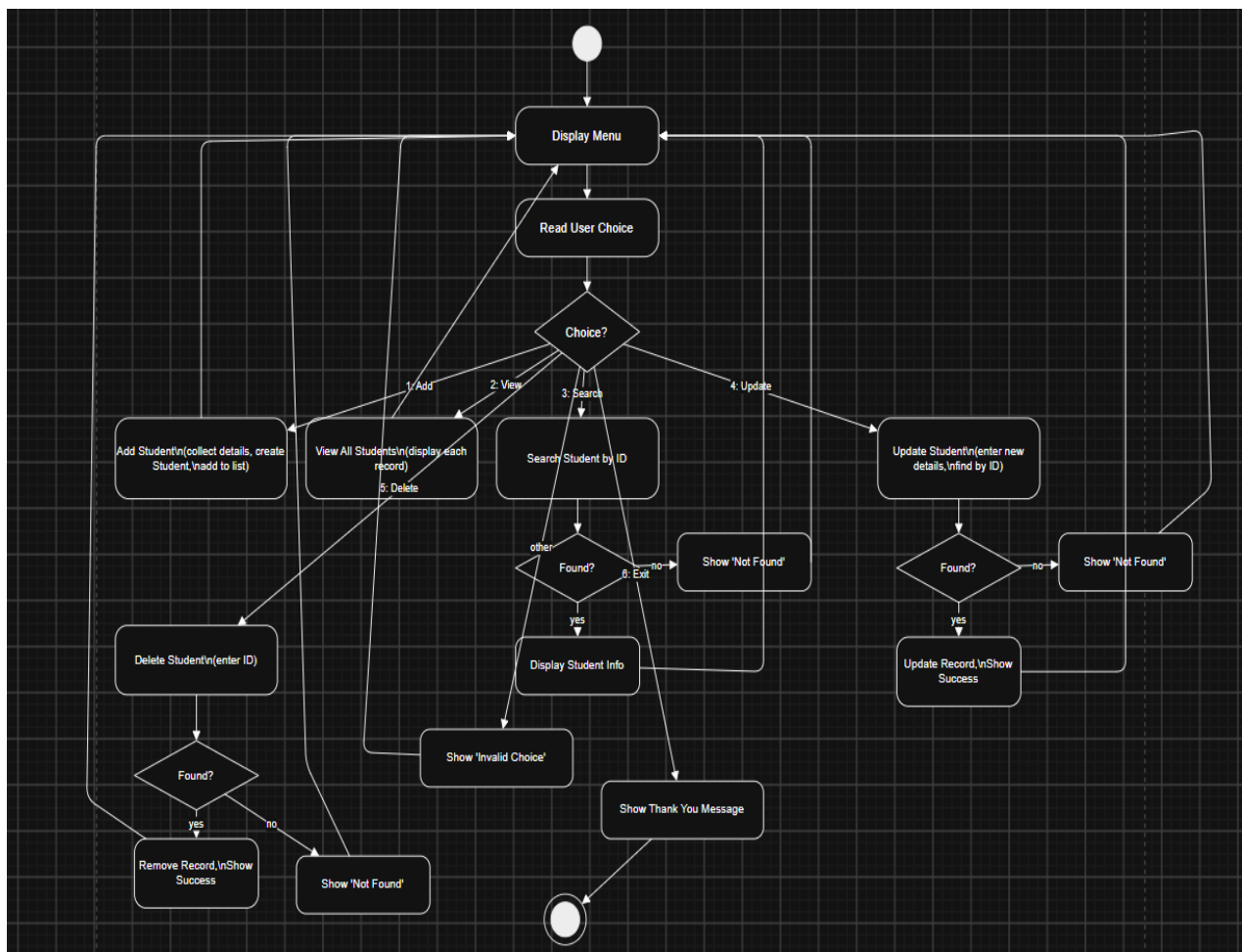
4 UML Diagrams



Class Diagram



Use case Diagram



Activity diagram

5. Identification of AI-Assisted Code

1. Duplicate Student ID Validation

Version A - The system added students directly without checking for duplicate IDs.

```
public void addStudent(Student student) { 1 usage
    students.add(student);
    System.out.println("Student added successfully!");
}
```

Version B (AI-Assisted)

AI suggested validating the Student ID before adding the student.

```
public boolean addStudent(Student student) { no usages

    if (searchStudent(student.getId()) != null) {
        return false; // Duplicate ID found
    }

    students.add(student);
    return true;
}
```

2. Age Validation

Version A

```
System.out.print("Enter Age: ");
int age = scanner.nextInt();
scanner.nextLine();
```

Version B

```
// Validate Age
int age;

while (true) {

    try {

        System.out.print("Enter Age: ");
        age = scanner.nextInt();

        if (age > 0) {
            scanner.nextLine();
            break;
        }

        System.out.println("Age must be greater than 0.");

    } catch (InputMismatchException e) {

        System.out.println("Invalid age. Please enter a number.");
        scanner.nextLine();

    }

}
```

3. GPA Validation

Version A

```
System.out.print("Enter GPA: ");
double gpa = scanner.nextDouble();
scanner.nextLine();
```

Version B

```
// Validate GPA
double gpa;

while (true) {

    try {

        System.out.print("Enter GPA (0.0 - 4.0): ");
        gpa = scanner.nextDouble();

        if (gpa >= 0 && gpa <= 4.0) {
            scanner.nextLine();
            break;
        }

        System.out.println("GPA must be between 0.0 and 4.0.");

    } catch (InputMismatchException e) {

        System.out.println("Invalid GPA.");
        scanner.nextLine();

    }

}
```

4. Exception Handling for Menu Selection

Version A

```
choice = scanner.nextInt();
scanner.nextLine();
```

Version B

```
// Validate menu input
try {
    choice = scanner.nextInt();
    scanner.nextLine();
} catch (InputMismatchException e) {
    System.out.println("Invalid input! Please enter a number.");
    scanner.nextLine();
    continue;
}
```

6 . AI prompts used during development

Prompt 1 - To generated the initial project structure.

Generate a Java Student Management System using object-oriented programming with CRUD operations.

Prompt 2 - Added input validation and prevented program crashes.

Improve the Student Management System by adding exception handling for invalid user input using try-catch blocks.

Prompt 3 - Improved data integrity.

Modify the program to prevent duplicate Student IDs from being added.

Prompt 4 - Improved documentation and code quality.

Refactor the Java code to improve readability, maintainability, and add meaningful comments without changing its functionality.

Task 3: Critical Analysis

1. How AI Changed My Coding Approach

AI-assisted programming significantly changed my approach. Instead of writing every line of code manually, I used AI to generate code suggestions, improve error handling and recommend better programming practices. This allowed me to spend more time understanding the program logic and testing the application rather than focusing on repetitive coding tasks.

AI also helped identify potential improvements such as input validation, exception handling and duplicate Student ID checking, which enhanced the overall quality of the application.

2. Did AI Improve or Reduce My Understanding?

AI generally improved my understanding of Java programming because it provided explanations alongside the generated code. By reviewing and modifying the AI-generated suggestions, I learned different approaches to solving programming problems and gained a better understanding of object-oriented programming concepts.

However, relying entirely on AI without analyzing the generated code could reduce a developer's understanding. Therefore, it is important to review, test and understand all AI-generated solutions before using them.

3. Situations Where AI Should Not Be Used

- Developing safety-critical systems such as medical or aviation software.
- Implementing highly sensitive security features.
- Writing software that requires strict legal or regulatory compliance.
- Solving complex business problems where detailed domain knowledge is required.
- Educational activities where students are expected to demonstrate their own programming skills.

4. Ethical considerations in AI-generated code

We are responsible for reviewing AI-generated code to ensure it is accurate, secure and appropriate for the application. AI-generated code may contain errors, security vulnerabilities or copyrighted material if used without proper verification. Additionally, students and professionals should use AI as a learning and productivity tool rather than presenting AI-generated work as entirely their own. Responsible use of AI promotes academic integrity, transparency and professional ethics.

Task 4: Conclusion

Artificial Intelligence is transforming software development by improving productivity, reducing development time and assisting developers with coding, debugging and documentation. In this project, both the traditional and AI-assisted versions of the Student Management System successfully achieved the required functionality.

The AI-assisted version demonstrated clear advantages in terms of code quality, error handling, readability and maintainability. Features such as input validation, exception handling and other things made the application more reliable and user-friendly.

But it should not replace a developer's knowledge and critical thinking. We must always review, test and understand AI-generated code before incorporating it into a software project. Therefore, AI should be considered an intelligent assistant that enhances software development rather than replacing human programmers.

GitHub Repository

The complete source code, project files, documentation, UML diagrams, and commit history are available at:

Repository Link:

https://github.com/hirushiwijenayaka-ai/Student_Management_System.git